

1

Analyzing Text Data with Deep Learning

Language is one of the most amazing abilities of human beings; it evolves during the individual's lifetime and is capable of conveying a message with complex meaning. Language in its natural form is not understandable to machines, and it is extremely challenging to develop an algorithm that can pick up the different nuances. Therefore, in this chapter, we will discuss how to represent text in a form that is digestible by machines.

In natural form, text cannot be directly fed to a **deep learning** model. In this chapter, we will discuss how text can be represented in a form that can be used by **machine learning** models. Starting with natural text, we will transform the text into numerical vectors that are increasingly sophisticated (one-hot encoding, **bag of words (BoW)**, **term frequency-inverse document frequency (TF-IDF)**) until we create vectors of real numbers that represent the meaning of a word (or document) and allow us to conduct operations (word2vec). In this chapter, we introduce deep learning models, such as **recurrent neural networks (RNNs)**, **long short-term memory (LSTM)**, **gated recurrent units (GRUs)**, and **convolutional neural network (CNNs)**, to analyze sequences and discuss their strengths as well as the problems associated with them. Finally, we will assemble these models all together to conduct text classification, showing the power of the learned approaches.

By the end of this chapter, we will be able to take a corpus of text and use deep learning to analyze it. These are the bases that will help us understand how a **large language model (LLM)** (such as ChatGPT) works internally.

In this chapter, we'll be covering the following topics:

- Representing text for AI

- Embedding, application, and representation
- RNNs, LSTMs, GRUs, and CNNs for text
- Performing sentiment analysis with embedding and deep learning

Technical requirements

In this chapter, we will use standard libraries for Python. The necessary libraries can be found within each of the Jupyter notebooks that are in the GitHub repository for this chapter: <https://github.com/PacktPublishing/Modern-AI-Agents/tree/main/chr1>. The code can be executed on a CPU, but a GPU is advised.

Representing text for AI

Compared to other types of data (such as images or tables), it is much more challenging to represent text in a digestible representation for computers, especially because there is no unique relationship between the meaning of a word (signified) and the symbol that represents it (signifier). In fact, the meaning of a word changes from the context and the author's intentions in using it in a sentence. In addition, native text has to be transformed into a numerical representation to be ingested by an algorithm, which is not a trivial task. Nevertheless, several approaches were initially developed to be able to find a vector representation of a text. These vector representations have the advantage that they can then be used as input to a computer.

First, a collection of texts (**corpus**) should be divided into fundamental units (words). This process requires making certain decisions and process operations that collectively are called **text normalization**. A sentence, therefore, is divided into words by exploiting the natural division of spaces (**text segmentation**); each punctuation mark is also considered a single word. In fact, punctuation marks are considered to be the boundaries of sentences and convey important information (change of topic, questions, exclamations).

The second step is the definition of what a word is and whether some terms in the corpus should be directly joined under the same vocabulary instance. For example, "He" and "he" represent the same instance; the former is only capitalized. Since an algorithm does not include such nuances, one must normalize the text in lowercase. In some cases, we want

to conduct more sophisticated normalizations such as **lemmatization** (joining words with the same root: “came” and “comes” are two forms of the verb) or **stemming** (stripping all suffixes of words).

Tokenization is the task of transforming a text into fundamental units. This is because, in addition to words, a text may also include percentages, numbers, websites, and other components. We will return to this later, but in the meantime, we will look at some simpler forms of tokenization.

One-hot encoding

In traditional **natural language processing (NLP)**, text representation is conducted using discrete symbols. The simplest example is one-hot encoding. From a sequence of text in a corpus (consisting of n different words), we obtain an n -dimensional vector. In fact, the first step is to compute the set of different words present in the whole text corpus called vocabulary. For each word, we obtain a vector as long as the size of the vocabulary. Then for each word, we will have a long vector composed mainly of zeros and ones to represent the word (one-hot vectors). This system is mainly used when we want a matrix of features and then train a model. This process is also called **vectorization**; here's a sparse vector for the following two words:

$$\textit{restaurant} = [0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0]$$

$$\textit{pizzeria} = [0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0]$$

There are different problems associated with this representation. First, it captures only the presence (or the absence) of a word in a document. Thus, we are losing all the semantic relationships between the words. Second, an average language has about 200,000 words, so for each word, we would have a vector of length 200,000. This leads to very sparse and high-dimensional vectors. For large corpora, we need high memory to store the vectors and high computational capacity to handle them. In addition, there is no notion of similarity. The two words in the preceding example are two places that sell food, and we would like the vectors representing these words to encode this similarity. If the vectors had a notion of similarity, we could conduct clustering, and the synonyms would be in the same cluster.

In order to obtain such a matrix, we must do the following:

- Standardize the text before tokenization. In this case, we simply transform everything into lowercase.
- We construct a vocabulary constituted of unique words and save the vocabulary so that in a case from a vector, we can get the corresponding word.
- We create an array and then populate it with **1** at the index of the word in the vocabulary; **0**s elsewhere.

Let's take a look at how this works in code:

```
import numpy as np
def one_hot_encoding(sentence):
    words = sentence.lower().split()
    vocabulary = sorted(set(words))
    word_to_index = {word: i for i,
                        word in enumerate(vocabulary)}
    one_hot_matrix = np.zeros((
        len(words), len(vocabulary)), dtype=int)
    for i, word in enumerate(words):
        one_hot_matrix[i, word_to_index[word]] = 1
    return one_hot_matrix, vocabulary
```

Let's look at a specific example:

```
sentence = "Should we go to a pizzeria or do you prefer a restaurant?"
one_hot_matrix, vocabulary = one_hot_encoding(sentence)
print("Vocabulary:", vocabulary)
print("One-Hot Encoding Matrix:\n", one_hot_matrix)
```

We get the following output:

```
Vocabulary: ['a', 'do', 'go', 'or', 'pizzeria', 'prefer', 'restaurant?', 'should', 't
o', 'we', 'you']
One-Hot Encoding Matrix:
[[0 0 0 0 0 0 0 0 1 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 1 0]
 [0 0 1 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 1 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 1 0 0 0 0 0 0 0]
 [0 0 0 1 0 0 0 0 0 0 0 0]
 [0 1 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0 0 0 0 1]
 [1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 1 0 0 0 0 0 0]
 [1 0 0 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 1 0 0 0 0 0]]
```

IMPORTANT NOTE

Observe how choosing another sentence will result in a different matrix and how, by increasing the length of the sentence, the matrix grows proportionally to the number of different words. Also, note that for repeated words, we get equal vectors. Check the preceding output.

Even if it is a simple method, we have obtained a first representation of text in a vectorial form.

Bag-of-words

In the previous section, we discussed one-hot encoding and some of the problems associated with this form of text representation. In the previous example, we worked with a single sentence, but a corpus is made up of thousands if not millions of documents; each of these documents contains several words with a different frequency. We want a system that preserves this frequency information, as it is important for the classification of text. In fact, documents that have similar content are similar, and their meaning will also be similar.

BoW is an algorithm for extracting features from text that preserves this frequency property. BoW is a very simple algorithm that ignores the position of words in the text and only considers this frequency property. The name “bag” comes precisely from the fact that any information concerning sentence order and structure is not preserved by the algorithm. For BoW, we only need a vocabulary and a way to be able to count words. In this case, the idea is to create document vectors: a single vector represents a document and the frequency of words contained in the vocabulary. *Figure 1.1* visualizes this concept with a few lines from *Hamlet*:

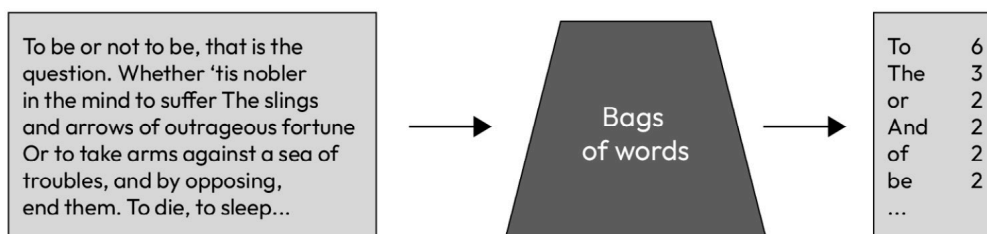


Figure 1.1 – Representation of the BoW algorithm

Even this representation is not without problems. Again, as the vocabulary grows, so will the size of the vectors (the size of each vector is equal to the length of the vocabulary). In addition, these vectors tend to be scattered, especially when the documents are very different from each other. High-dimensional or sparse vectors are not only problematic for memory and computational costs but for algorithms as well (the longer the vec-

tors, the more weight you need in the algorithm, leading to a risk of overfitting). This is called the **curse of dimensionality**; the greater the number of features, the less meaningful the distances between examples. For large corpora, some solutions have been proposed, such as ignoring punctuation, correcting misspelled words, stemming algorithms, or ignoring words with high frequency that don't add information (articles, prepositions, and so on).

In order to get a BoW matrix for a list of documents, we need to do the following:

- Tokenize each document to get a list of words.
- Create our vocabulary of unique words and map each word to the corresponding index in the vocabulary.
- Create a matrix where each row represents a document and each column, instead, a word in the vocabulary (the documents are the examples, and the words are the associated features).

Let's look at the code again:

```
import numpy as np
def bag_of_words(sentences):
    """
    Creates a bag-of-words representation of a list of documents.
    """
    tokenized_sentences = [
        sentence.lower().split() for sentence in sentences
    ]
    flat_words = [
        word for sublist in tokenized_sentences for word in sublist
    ]
    vocabulary = sorted(set(flat_words))
    word_to_index = {word: i for i, word in enumerate(vocabulary)}
    bow_matrix = np.zeros((
        len(sentences), len(vocabulary)), dtype=int)
    for i, sentence in enumerate(tokenized_sentences):
        for word in sentence:
            if word in word_to_index:
                bow_matrix[i, word_to_index[word]] += 1
    return vocabulary, bow_matrix
```

Here's an example:

```
corpus = ["This movie is awesome awesome",
          "I do not say is good, but neither awesome",
          "Awesome? Only a fool can say that"]
vocabulary, bow_matrix = bag_of_words(corpus)
print("Vocabulary:", vocabulary)
print("Bag of Words Matrix:\n", bow_matrix)
```

This prints the following output:

```
Vocabulary: ['a', 'awesome', 'awesome?', 'but', 'can', 'do', 'fool', 'good,', 'i', 'i', 's', 'movie', 'neither', 'not', 'only', 'say', 'that', 'this']
Bag of Words Matrix:
[[0 2 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1]
 [0 1 0 1 0 1 0 1 1 1 0 1 1 0 1 0 0]
 [1 0 1 0 1 0 1 0 0 0 0 0 0 1 1 1 0]]
```

IMPORTANT NOTE

Note how in the example, the word “awesome” is associated with a review with a positive, neutral, or negative meaning. Without context, the frequency of the word “awesome” alone does not tell us the sentiment of the review.

Here, we have learned how to transform text in a vectorial form while keeping the notion of frequency for each word.

TF-IDF

In the previous section, we obtained a document-term matrix. However, the raw frequency is very skewed and does not always allow us to discriminate between two documents. The document-term matrix was born in information retrieval to find documents, though words such as “good” or “bad” are not very discriminative since they are often used in text with a generic meaning. In contrast, words with low frequency are much more informative, so we are interested more in relative than absolute frequency:



Figure 1.2 – Intuition of the components of TF-IDF

Instead of using raw frequency, we can use the logarithm in base 10, because a word that occurs 100 times in a document is not 100 times more relevant to its meaning in the document. Of course, since vectors can be very sparse, we assign 0 if the frequency is 0. Second, we want to pay more attention to words that are present only in some documents. These words will be more relevant to the meaning of the document, and we want to preserve this information. To do this, we normalize by IDF. IDF is defined as the ratio of the total number of documents in the corpus to how many documents a term is present in. To summarize, to obtain the TF-IDF, we multiply TF by the logarithm of IDF.

This is demonstrated in the following code block:

```
import numpy as np

def compute_tf(sentences):
    """Compute the term frequency matrix for a list of sentences."""
    vocabulary = sorted(set(
        word for sentence in sentences
        for word in sentence.lower().split()))
    word_index = {word: i for i, word in enumerate(vocabulary)}
    tf = np.zeros((
        len(sentences), len(vocabulary)), dtype=np.float32)
    for i, sentence in enumerate(sentences):
        words = sentence.lower().split()
        word_count = len(words)
        for word in words:
            if word in word_index:
                tf[i, word_index[word]] += 1 / word_count
    return tf, vocabulary

def compute_idf(sentences, vocabulary):
    """Compute the inverse document frequency for a list of sentences"""
    num_documents = len(sentences)
    idf = np.zeros(len(vocabulary), dtype=np.float32)
    word_index = {word: i for i, word in enumerate(vocabulary)}
    for word in vocabulary:
        df = sum(
            1 for sentence in sentences
            if word in sentence.lower().split()
        )
        idf[word_index[word]] = np.log(
            num_documents / (1 + df)) + 1 # Smoothing
    return idf

def tf_idf(sentences):
    """Generate a TF-IDF matrix for a list of sentences."""
```

```

tf, vocabulary = compute_tf(sentences)
idf = compute_idf(sentences, vocabulary)
tf_idf_matrix = tf * idf
return vocabulary, tf_idf_matrix
vocabulary, tf_idf_matrix = tf_idf(corpus)
print("Vocabulary:", vocabulary)
print("TF-IDF Matrix:\n", tf_idf_matrix)

```

This generates the following output:

```

Vocabulary: ['a', 'awesome', 'awesome?', 'but', 'can', 'do', 'fool', 'good,', 'i', 'i
s', 'movie', 'neither', 'not', 'only', 'say', 'that', 'this']
TF-IDF Matrix:
[[0.      0.4      0.      0.      0.
 0.      0.      0.      0.2      0.28109303 0.
 0.      0.      0.      0.      0.28109303]
[0.      0.11111111 0.      0.1561628 0.      0.1561628
 0.      0.1561628 0.1561628 0.11111111 0.      0.1561628
 0.1561628 0.      0.11111111 0.      0.      ]
[0.20078073 0.      0.20078073 0.      0.20078073 0.
 0.20078073 0.      0.      0.      0.
 0.      0.20078073 0.14285715 0.20078073 0.      ]]

```

IMPORTANT NOTE

In this example, we used the same corpus as in the previous section. Note how word frequencies changed after this normalization.

In this section, we learned how we can normalize text to decrease the impact of the most frequent words and give relevance to words that are specific to a subset of documents. Next, we'll discuss embedding.

Embedding, application, and representation

In the previous section, we discussed how to use vectors to represent text. These vectors are digestible for a computer, but they still suffer from some problems (sparsity, high dimensionality, etc.). According to the distributional hypothesis, words with a similar meaning frequently appear close together (or words that appear often in the same context have the same meaning). Similarly, a word can have a different meaning depending on its context: “I went to deposit money in the *bank*” or “We went to do a picnic on the river *bank*.” In the following diagram, we have a high-level representation of the embedding process. So, we want a process that allows us to start from text to obtain an array of vectors, where each vector corresponds to the representation of a word. In this case, we want a model that will then allow us to map each word to a vector representa-

tion. In the next section, we will describe the process in detail and discuss the theory behind it.

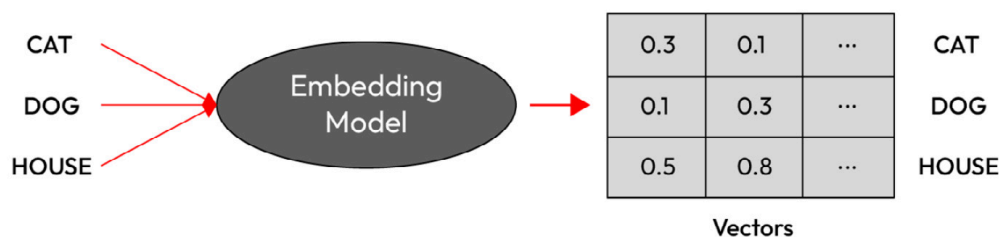


Figure 1.3 – High-level representation of the embedding process

We would, therefore, like to generate vectors that are small in size, composed of real (dense) numbers, and that preserve this contextual information. Thus, the purpose is to have vectors of limited size that can represent the meaning of a word. The scattered vectors we obtained earlier cannot be used efficiently for mathematical operations or downstream tasks. Also, the more words there are in the vocabulary, the larger the size of the vectors we get. Therefore, we want dense vectors (with real numbers) that are small in size and whose size does not increase as the number of words in the vocabulary increases.

In addition, these vectors have a distributed representation of the meaning of the word (whereas in sparse vectors, it was local or where the **1** was located). As we will see a little later, these dense vectors can be used for different operations because they better represent the concept of similarity between words. These dense vectors are called **word embeddings**.

This concept was introduced in 2013 by Mikolov with a framework called **word2vec**, which will be described in detail in the next section.

Word2vec

The intuition behind word2vec is simple: predict a word w from its context. To do this, we need a **neural network** and a large corpus. The revolutionary idea is that by training this neural network to predict which words c are needed near the target word w , the weights of the neural network will be the embedding vectors. This model is self-supervised; the labels in this case are implicit, and we do not provide them.

Word2vec simplifies this idea by making the system extremely fast and effective in two ways: by turning the task into binary classification (Is the word c needed in the context of the word w ? Yes or no?) and using a logistic regression classifier:

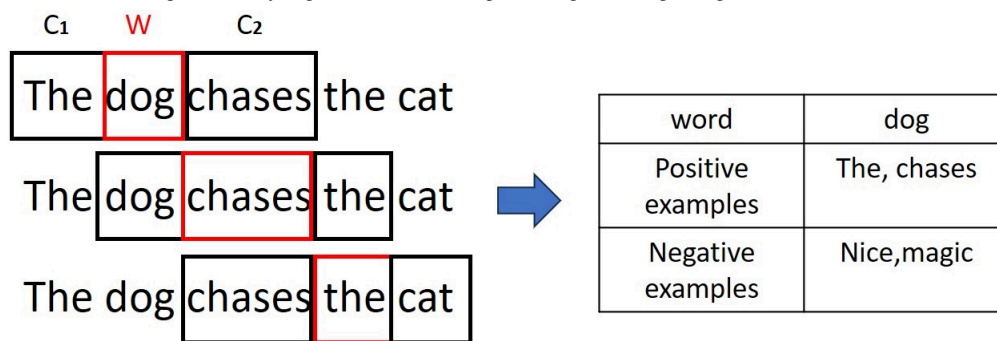


Figure 1.4 – In word2vec, we slide a context window (here represented as a three-word context window), and then we randomly sample some negative words

Given a text t , we scroll a window c (our context) for a word w in the center of our window; the words around it are examples of the positive class. After that, we select other random words as negative examples. Finally, we train a model to classify the positive and negative examples; the weights of the model are our embeddings.

Given a word w and a word c , we want the probability that the word c is in the context of w to be similar to its embedding similarity. In other words, if the vector representing w and c are similar, c must often be in the context of w (word2vec is based on the notion of context similarity). We define this embedding similarity by the dot product between the two embedding vectors for w and c (we use the sigmoid function to transform this dot product into a probability and thus allow comparison). So, the probability that c is in the context of w is equal to the probability that their embeddings are similar:

$$P + |w, c) = \sigma(c \bullet w) = \frac{1}{1 + \exp(-c \bullet w)}$$

This is done for all words in context L . To simplify, we assume that all words in the context window are independent, so we can multiply the probabilities of the various words c . Similarly, we want to ensure that this dot product is minimal for words that are not in the context of word w . So, on the one hand, we maximize the probability for words in the context, and on the other hand, we minimize the probability for words that are not in the context. In fact, words that are not in the context of w are randomly extracted during training, and the process is the same:

$$P + |w, c) = \prod_{i=1}^L \frac{1}{1 + \exp(-c_i \bullet w)}$$

$$P(w|c) = \prod_{i=1}^L \frac{1}{1 + \exp(-c_i \cdot w)}$$

For simplicity, we take the logarithm of probability:

$$\log P(w|c) = \sum_{i=1}^L \log \left(\frac{1}{1 + \exp(-c_i \cdot w)} \right)$$

$$-\log P(w|c) = \sum_{i=1}^L \log \left(1 + \exp(-c_i \cdot w) \right)$$

The matrix of weights w is our embedding; it is what we will use from now on. Actually, the model learns two matrices of vectors (one for w and one for c), but the two matrices are very similar, so we take just one. We then use cross-entropy to train the models and learn the weights for each vector:

$$LCE = -\log P(w|c_{pos}) - \sum_{i=1}^L \log P(w|c_{neg})$$

This is represented visually in the following diagram:

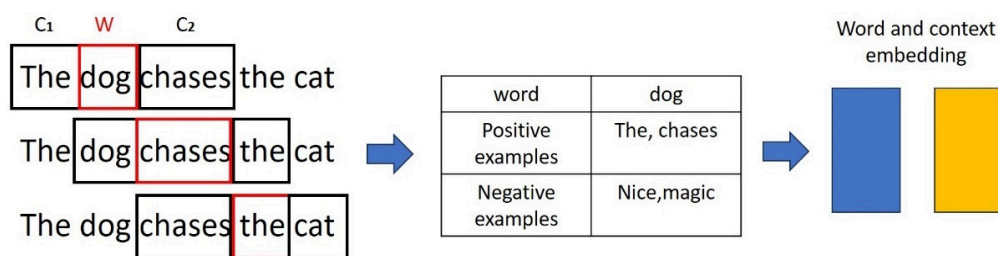


Figure 1.5 – Word and context embedding

The following choices affect the quality of embedding:

- *Data quality is critical.* For example, leveraging Wikipedia allows better embedding for semantic tasks, while using news improves performance for syntactic tasks (a mixture of the two is recommended). Using Twitter or other social networks can insert bias.
- *At the same time, a larger amount of text improves embedding performance.* A large amount of text can partially compensate for poor quality but at the cost of much longer training (for example, Common Crawl is a huge dataset downloaded from the internet that is pretty dirty, though).
- *The number of dimensions is another important factor.* The larger the size of the embedding, the better its performance. 300 is considered a

sweet spot because, beyond this size, number performance does not increase significantly.

- *The size of the context window also has an impact.* Generally, a context window of 4 is used, but a context window of 2 allows for vectors that better identify parts of speech. In contrast, long context windows are more useful if we are interested in similarity more broadly.

In Python, we can easily get an embedding from lists of tokens using the following code:

```
from gensim.models import Word2Vec
model = Word2Vec(sentences=list_of_tokens,
                  sg=1,
                  vector_size=100,
                  window=5,
                  workers=4)
```

IMPORTANT NOTE

The complete code is present in the GitHub repository. We used an embedding of 100 dimensions and a window of 5 words.

Once we have our embedding, we can visualize it. For example, if we try **clustering** the vectors of some words, words that have similar meanings should be closer together:

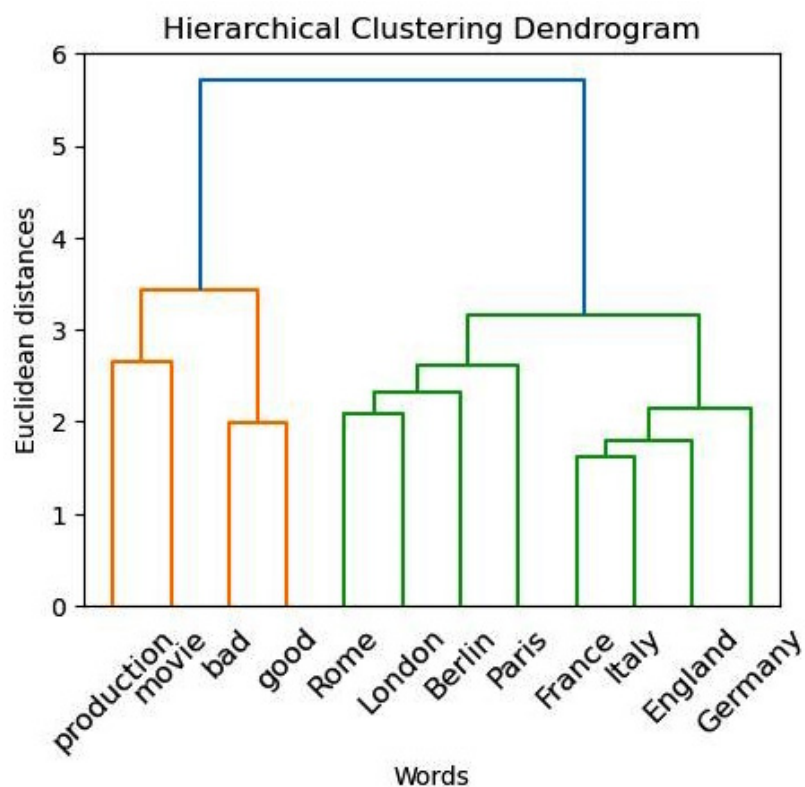


Figure 1.6 – Clustering of some of the vectors obtained from the embedding

Another way to visualize vectors is to use dimensionality reduction techniques. Vectors are multidimensional (100-1,024), so it is more convenient to reduce them to two or three dimensions so that they can be visualized more easily. Some of the most commonly used techniques are **principal component analysis (PCA)** and **t-distributed stochastic neighbor embedding (t-SNE)**. **Uniform Manifold Approximation and Projection (UMAP)**, on the other hand, is a technique that has become the first choice for visualizing multidimensional data in recent years:

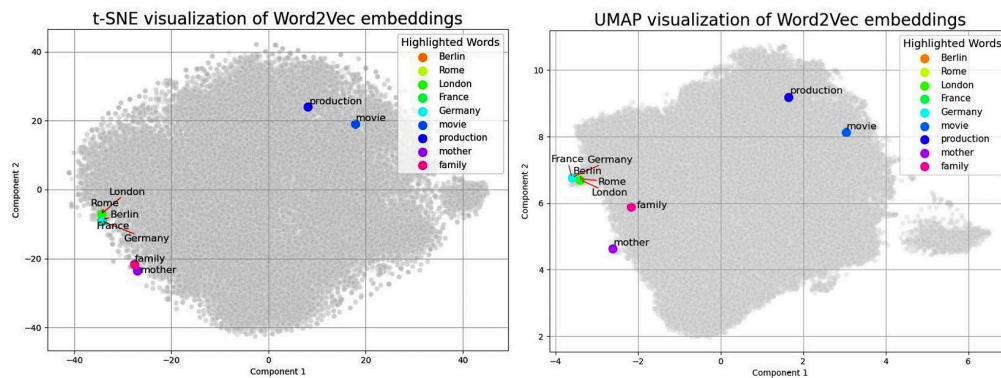


Figure 1.7 – 2D projection of word2vec embedding highlighting some examples

UMAP has emerged because it produces visualizations that better preserve semantic meaning and relationships between examples and also better represent local and global structures. This makes for better clusters, and UMAP can also be used in preprocessing steps before a classification task on vectors.

A notion of similarity for text

Once we have obtained vector representations, we need a method to calculate the similarity between them. This is crucial in many applications—for instance, to find words in an embedding space that are most similar to a given word, we compute the similarity between its vector and those of other words. Similarly, given a query sentence, we can retrieve the most relevant documents by comparing its vector with document embeddings and selecting those with the highest similarity.

Most similarity measures are based on the **dot product**. This is because the dot product is high when the two vectors have values in the same dimension. In contrast, vectors that have zero alternately will have a dot

product of zero, thus orthogonal or dissimilar. This is why the dot product was used as a similarity measure for word co-occurrence matrices or with vectors derived from document TF matrices:

$$\text{dotproduct} : \mathbf{a} \bullet \mathbf{b} = \sum_{i=1}^N a_i \times b_i = a_1 \times b_1 + a_2 \times b_2 + \cdots a_n \times b_n$$

$$\text{magnitude} = |\mathbf{a}| = \sqrt{\sum_{i=1}^N a_i^2}$$

The dot product has several problems, though:

- It tends to favor vectors with long dimensions
- It favors vectors with high values (which, in general, are those of very frequent and, therefore, useless words)
- The value of the dot product has no limits

Therefore, alternatives have been sought, such as a normalized version of the dot product. The normalized dot product is equivalent to the cosine of the angle between the two vectors, hence **cosine similarity**:

$$\cos\Theta = \frac{\mathbf{a} \bullet \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} = \frac{\sum_{i=1}^N a_i \times b_i}{\sqrt{\sum_{i=1}^N a_i^2} \sqrt{\sum_{i=1}^N b_i^2}}$$

Cosine similarity has some interesting properties:

- It is between -1 and 1. Opposite or totally dissimilar vectors will have a value of -1, 0 for orthogonal vectors (or totally dissimilar for scattered vectors), and 1 for perfectly similar vectors. Since it measures the angle between two vectors, the interpretation is easier and is within a specific range, so it allows one intuitively to understand the similarity or dissimilarity.
- It is fast and cheap to compute.
- It is less sensitive to word frequency and, thus, more robust to outliers.
- It is scale-invariant, meaning that it is not influenced by the magnitude of the vectors.
- Being normalized, it can also be used with high-dimensional data.

For 2D vectors, we can plot to observe these properties:

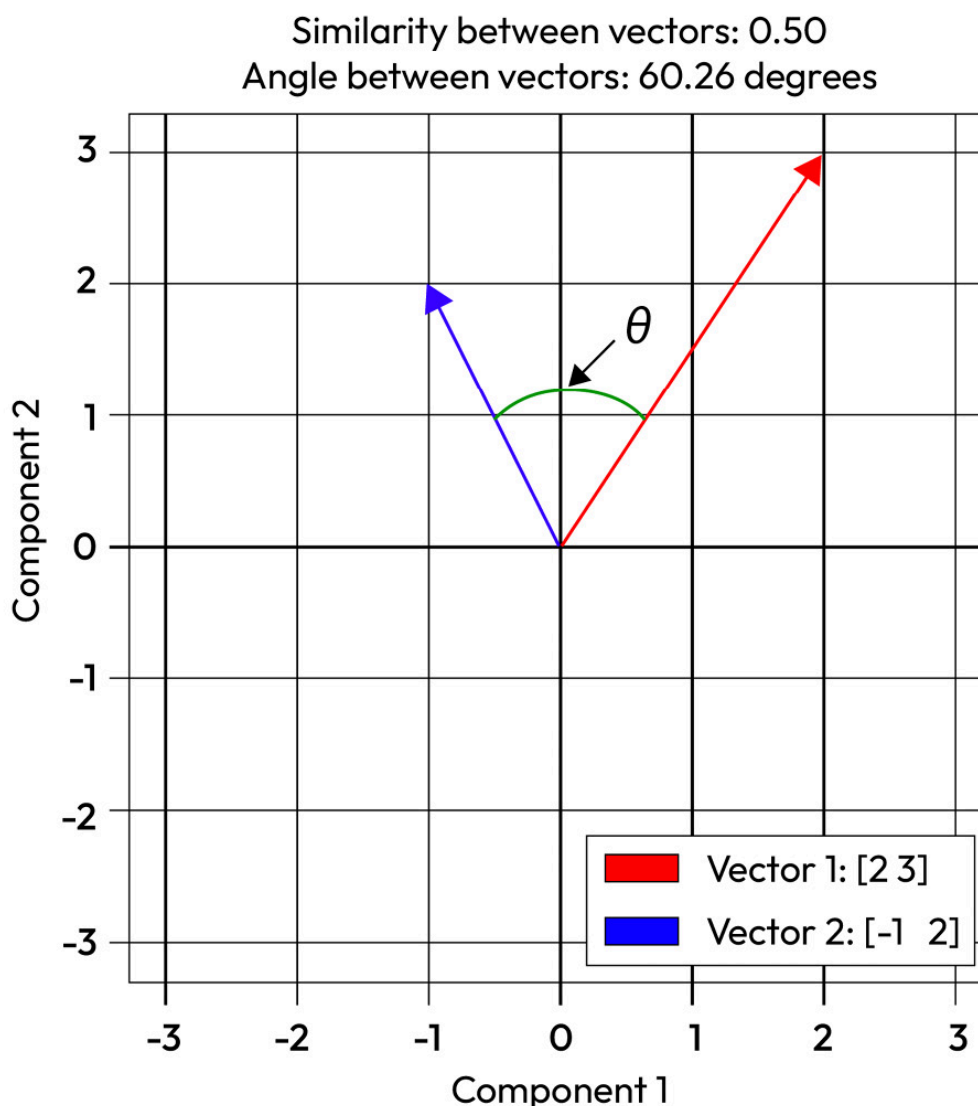


Figure 1.8 – Example of cosine similarity between two vectors

In the next section, we can define the properties of our trained embedding using this notion of similarity.

Properties of embeddings

Embeddings are a surprisingly flexible method and manage to encode different syntactic and semantic properties that can both be visualized and exploited for different operations. Once we have a notion of similarity, we can search for the words that are most similar to a word w . Note that similarity is defined as appearing in the same context window; the model cannot differentiate synonyms and antonyms.

In addition, the model is also capable of representing grammatical relations such as superlatives or verb forms.

Another interesting relationship we can study is analogies. The parallelogram model is a system for representing analogies in a cognitive space.

The classic example is *king:queen::man:?* (which in a formula would be *a:b::a*:?*). Given that we have vectors, we can turn this into an *a-a*+b* operation.

We can test this in Python using the embedding model we have trained:

- We can check the most similar words
- We can test the analogy
- We can then test the capacity to identify synonyms and antonyms

The code for this process is as follows:

```
word_1 = "good"
syn = "great"
ant = "bad"
most_sim = model.wv.most_similar("good")
print("Top 3 most similar words to {} are :{}".format(
    word_1, most_sim[:3]))
synonyms_dist = model.wv.distance(word_1, syn)
antonyms_dist = model.wv.distance(word_1, ant)
print("Synonyms {}, {} have cosine distance: {}".format(
    word_1, syn, synonyms_dist))
print("Antonyms {}, {} have cosine distance: {}".format(
    word_1, ant, antonyms_dist))
a = 'king'
a_star = 'man'
b = 'woman'
b_star = model.wv.most_similar(positive=[a, b], negative=[a_star])
print("{} is to {} as {} is to: {} ".format(
    a, a_star, b, b_star[0][0]))
```

IMPORTANT NOTE

This is done with the embedding we have trained before. Notice how the model is not handling antonyms well.

The method is not exactly perfect, so sometimes the right answer is not the first result, but it could be among the first three inputs. Also, this system works for entities that are frequent within the text (city names, common words) but much less with rarer entities.

Embeddings can also be used as a tool to study how the meaning of a word changes over time, especially if you have text corpora that span sev-

eral decades. This is demonstrated in the following diagram, which shows how the meanings of the words *gay*, *broadcast*, and *awful* have changed:

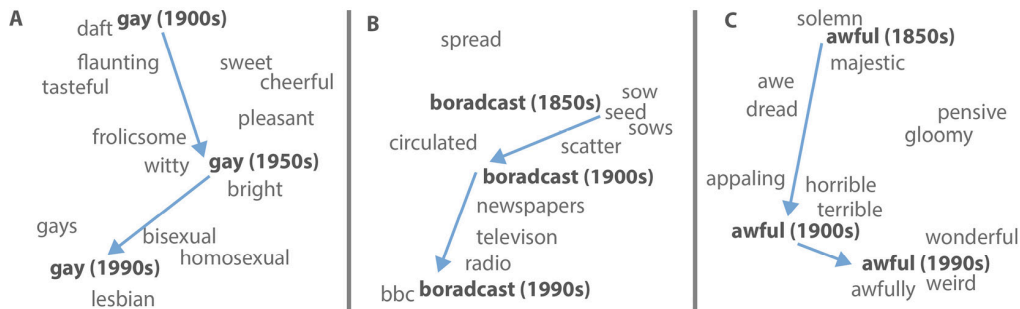


Figure 1.9 – 2D visualization of how the semantic meaning of a word changes over the years; these projections are obtained using text from different decades and embeddings (<https://arxiv.org/abs/1605.09096>)

Finally, a word can still have multiple meanings! For example, common words such as “good” have more than one meaning depending on the context. One may wonder whether a vector for a word in an embedding represents only one meaning or whether it represents the set of meanings of a word. Fortunately, embedding vectors represent a weighted sum of the various meanings of a word (linear superposition). The weights of each meaning are proportional to the frequency of that meaning in the text. Although these meanings reside in the same vector, when we add or subtract during the calculation of analogies, we are working with these components. For example, “apple” is both a fruit and the name of a company; if we conduct the operation *apple:red::banana:?*, we are subtracting only a very specific semantic component from the apple vector (the component that is similar to red). This flexibility can be useful when we want to disambiguate meanings. Also, since the vector space is sparse, by exploiting sparse coding, we can separate the various meanings:

tie					spring				
trousers	season	scoreline	wires	operatic	beginning	dampers	flower	creek	humid
blouse	teams	goalless	cables	soprano	until	brakes	flowers	brook	winters
waistcoat	winning	equaliser	wiring	mezzo	months	suspension	flowering	river	summers
skirt	league	clinchng	electrical	contralto	earlier	absorbers	fragrant	fork	ppen
sleeved	finished	scoreless	wire	baritone	year	wheels	lilies	piney	warm
pants	championship	replay	cable	coloratura	last	dampers	flowered	elk	temperatures

Figure 1.10 – Table showing how a vector in word2vec is encoding for different meanings at the same time (<https://aclanthology.org/Q18-1034/>)

These vectors are now providing contextual and semantic meaning for each word in the text. We can use this rich source of information for tasks such as text classification. What we need now are models that can handle

the sequential nature of text, which we will learn about in the next section.

RNNs, LSTMs, GRUs, and CNNs for text

So far, we have discussed how to represent text in a way that is digestible for the model; in this section, we will discuss how to analyze the text once a representation has been obtained. Traditionally, once we obtained a representation of the text, it was fed to models such as naïve Bayes or even algorithms such as logistic regression. The success of neural networks has made these machine learning algorithms outdated. In this section, we will discuss deep learning models that can be used for various tasks.

RNNs

The problem with classical neural networks is that they have no memory. This is especially problematic for time series and text inputs. In a sequence of words t , the word w at time t depends on the w at time $t-1$. In fact, in a sentence, the last word is often dependent on several words in the sentence. Therefore, we want an NN model that maintains a memory of previous inputs. An **RNN** maintains an internal state that maintains this memory; that is, it stores information about previous inputs, and the outputs it produces are affected by previous inputs. These networks perform the same operation on all elements of the sequence (hence recurrent) and maintain the memory of this operation:

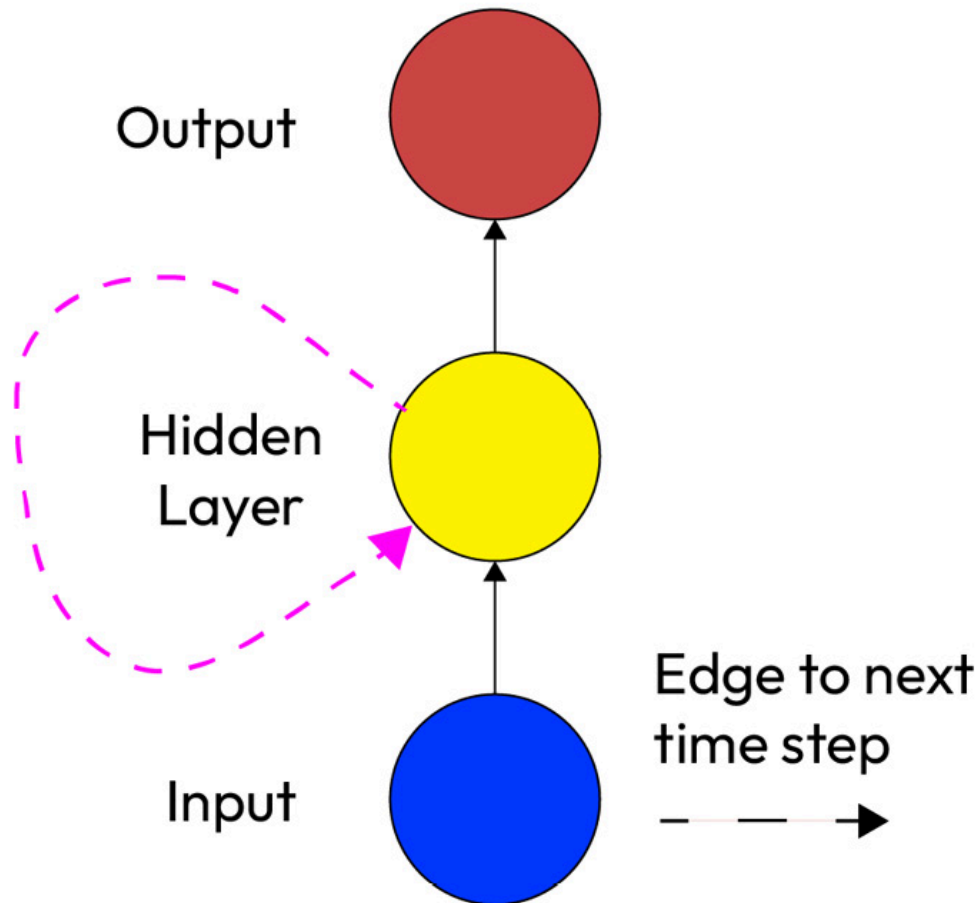


Figure 1.11 – Simple example of an RNN

(<https://arxiv.org/pdf/1506.00019>)

A classical neural network (**feedforward neural network**) considers inputs to be independent, and one layer of a neural network performs the following operation for a vector representing the element at time t :

$$y(t) = \sigma(Wx^t + b)$$

However, in a simple RNN, the following operations are conducted:

$$a^t = b + Uh^{t-1} + Wx^t$$

$$h^t = \tanh(a^t)$$

$$o^t = c + Vh^t$$

$$y^t = \sigma(o^t)$$

These operations may seem complicated, but in fact, we are simply maintaining a hidden state that considers the previous iterations. The first equation is a normal feedforward layer modified, in which we multiply the previously hidden state h by a set of weights U . This matrix U allows

us to control how the neural network uses the previous context to bind input and past inputs (how the past influences the output for input at time t). In the second equation, we create a new hidden state that will then be used for subsequent computations but also for the next input. In the third equation, we are creating the output; we use a bias vector and a matrix to compute the output. In the last equation, it is simply passed as a nonlinearity function.

These RNNs can be seen as unrolled entities throughout time, in which we can represent the network and its computations throughout the sequence:

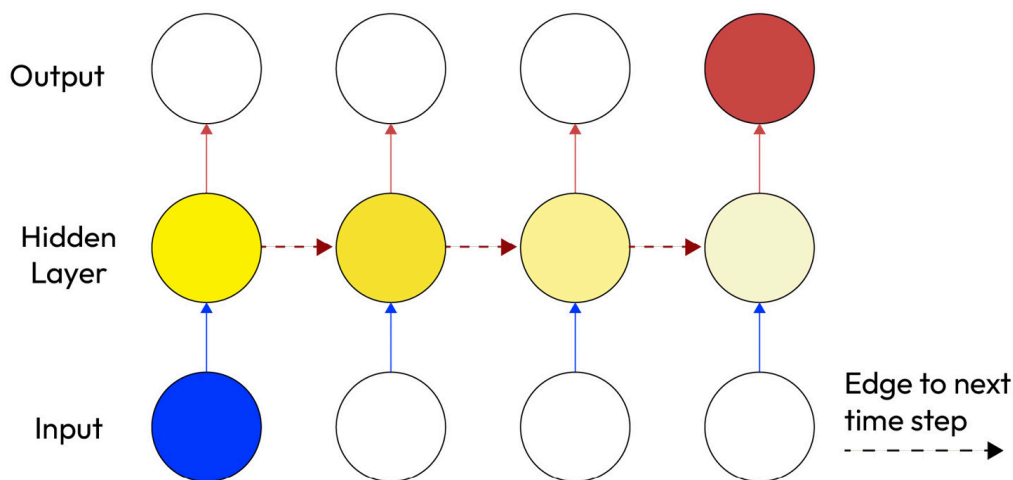


Figure 1.12 – Simple example of an RNN unrolled through the sequence (<https://arxiv.org/pdf/1506.00019>)

We can test in Python with a PyTorch RNN to see how it is transforming the data:

```
array = np.random.random((10, 5, 3))
# Convert the numpy array to a PyTorch tensor
data_tensor = torch.tensor(array, dtype=torch.float32)
RNN = nn.RNN(input_size=3, hidden_size=10,
              num_layers=1, batch_first=True)
output, hidden = RNN(data_tensor)
output.shape
```

IMPORTANT NOTE

Notice how the model is transforming the data; we can also access the hidden state.

We can see several interesting things:

- The RNN is not limited by the size of the input; it is a cyclic operation that is conducted over the entire sequence. RNNs basically process one word at a time. This cyclicity also means backpropagation is conducted for each time step. Although this model works well for series analysis, its sequential nature does not allow it to be parallelized.
- Theoretically, this model could be trained with infinite sequences of words; theoretically, after a few time steps, it begins to forget the initial inputs.
- Training can become inefficient due to the vanishing gradient problem, where gradients must propagate from the final cell back to the initial one. During this process, they can shrink exponentially and approach zero, making it difficult for the model to learn long-range dependencies. Conversely, the exploding gradient problem can also occur, where gradients grow uncontrollably large, leading to unstable training.

RNNs are not the only form of deep learning models that are relevant to this topic.

LSTMs

In theory, RNNs should be able to process long sequences and remember the initial input. However, in reality, the information inside the hidden state is local rather than global, and for a time t , it considers only the previous time steps and not the entire sequence. The main problem with such a simple model is that the hidden state must simultaneously fulfill two roles: provide information relevant to the output at time t and store memory for future decisions.

An **LSTM** is an extension of RNNs, designed with the idea that the model can forget information that is not important and keep only the important context.

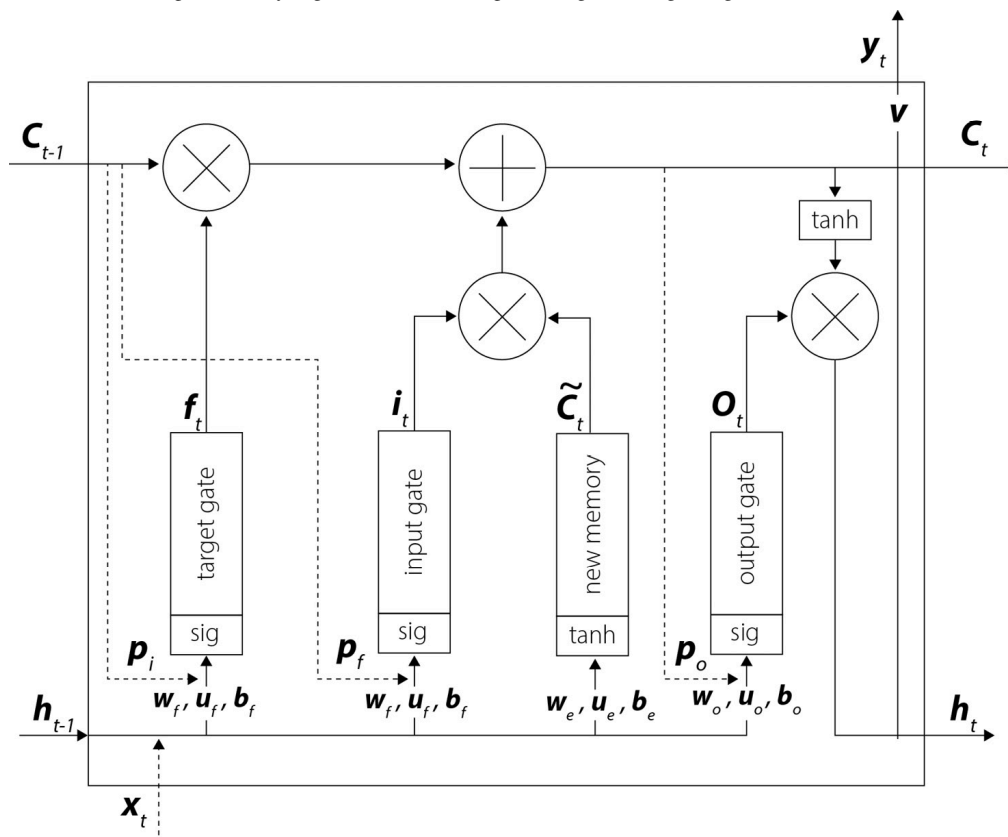


Figure 1.13 – Internal structure of an LSTM cell

(<https://arxiv.org/pdf/2304.11461>)

An LSTM has internal mechanisms to control the information (gates) within the layer; additionally, it has a dedicated context layer. So, we have two hidden states in which the first, h , serves for information at time t (short memory) and the other, c , for information at long term. The gates can be open (1) or closed (0); this is achieved by a feed-forward layer with sigmoid activation to squeeze values between zero and one. After that, we use the **Hadamard product** (or pointwise multiplication) for the gating mechanism of a layer. This multiplication acts as a binary gate, allowing information to pass if the value is close to 1 or blocking it when the value is close to 0. These gates allow a dynamic system in which during a time step, we decide how much information we preserve and how much we forget.

The first gate is called the **forget gate** because it is used to forget information that is no longer needed from the context and, therefore, will no longer be needed in the next time step. So, we will use the output of the forget gate to multiply the context. At this time, we extract both information from the input and the previous time step's hidden state. Each gate has a set of gate-specific U weights:

$$f^t = \sigma(b_f + U_f h^{t-1} + W_f x^t)$$

$kt = ct - 1 \odot ft$

The next step is to extract information from the input and decide which of that information will be added to the context. This is controlled by an **input gate** i that controls how much information will then be added. The context is then obtained by the sum of what we add and what we forget:

$$g^t = \tanh(b_g + U_g h^{t-1} + W_g x^t)$$

$$i^t = \sigma(b_i + U_i h^{t-1} + W_i x^t)$$

$jt = g(t) \odot it$

$ct = j(t) + kt$

The final step is the calculation of the output; this is achieved with a final gate. The output or final layer decision is also used to update the hidden state:

$$o^t = \sigma(b_o + U_o h^{t-1} + W_o x^t)$$

$kt = ot \odot \tanh(ct)$

These gates are independent of each other so that efficient implementations of the LSTM can parallelize them. We can test in Python with a PyTorch RNN to see how it is transforming the data:

```
data_tensor = torch.tensor(np.random.random((10, 5, 3)),
                           dtype=torch.float32)

LSTM = nn.LSTM(input_size=3, hidden_size=10,
               num_layers=1, batch_first=True)
output, (hidden, cell) = LSTM(data_tensor)
output.shape
```

IMPORTANT NOTE

Notice how the model is transforming the data; we can access the hidden state as well as the cell state.

We can also note some other interesting properties:

- Computation augmentation is internal to layers, which means we can easily substitute LSTMs for RNNs.
- The LSTM manages to preserve information for a long time because it retains only the relevant part of the information and forgets what is not needed.
- Standard practice is to initialize an LSTM with vector 1 (preserves everything), after which it learns by itself what to forget and what to add.
- An LSTM, as opposed to an RNN, can remember up to 100 time steps (the RNN after 7 time steps starts forgetting). The plus operation makes vanishing or exploding gradients less likely.

Let's look at another model option that is computationally lighter but still has this notion of context vector.

GRUs

GRUs are another variant of RNNs to solve the vanishing gradient problem, thus making them more effective in remembering information. They are very similar to LSTMs since they have internal gates, but they are much simpler and lighter. Despite having fewer parameters, GRUs can converge faster than LSTMs and still achieve comparable performance. GRUs exploit some of the elements that have made LSTMs so effective: the plus operation, the Hadamard product, the presence of a context, and the control of information within the layer:



At this point, we have the final update of the hidden state in which we also use the update gate:

$$\mathbf{h}^t = \mathbf{z}^t \odot \mathbf{h}^{t-1} + (1 - \mathbf{z}^t) \odot \overline{\mathbf{h}}^t$$

We can test in Python with a PyTorch RNN to see how it is transforming the data:

```
data_tensor = torch.tensor(np.random.random((10, 5, 3)),
                           dtype=torch.float32)
GRU = nn.GRU(input_size=3, hidden_size=10,
              num_layers=1, batch_first=True)
output, hidden = GRU(data_tensor)
output.shape
```

IMPORTANT NOTE

Notice how the model is transforming the data; we can also access the hidden state.

We can see some interesting elements here as well:

- GRU networks are similar to LSTM networks, but they have the advantage of fewer parameters and are computationally more efficient. This means, though, they are more prone to overfitting.
- They can handle long sequences of data without forgetting previous inputs. For many textual tasks (but also speech recognition and music generation) they perform quite well, though they are less efficient than LSTMs when it comes to modeling long-term dependencies or complex patterns.

Next, we'll look at CNNs.

CNNs for text

CNNs are designed to find patterns in images (or other 2D matrixes) by running a filter (a matrix or kernel) along them. The convolution is conducted pixel by pixel, and the filter values are multiplied by the pixels in the image and then summed. During training, a weight is learned for each of the filter entries. For each filter, we get a different scan of the image that can be visualized; this is called a feature map.

Convolutional networks have been successful in **computer vision** because of their ability to extract local information and recognize complex patterns. For this reason, convolutional networks have been proposed for sequences. In this case, 1-dimensional convolutional networks are exploited, but the idea is the same. In fact, on a sequence, 1D convolution is used to extract a feature map (instead of being a 2-dimensional filter or matrix, we have a uni-dimensional filter that can be seen as the context window of word2vec):

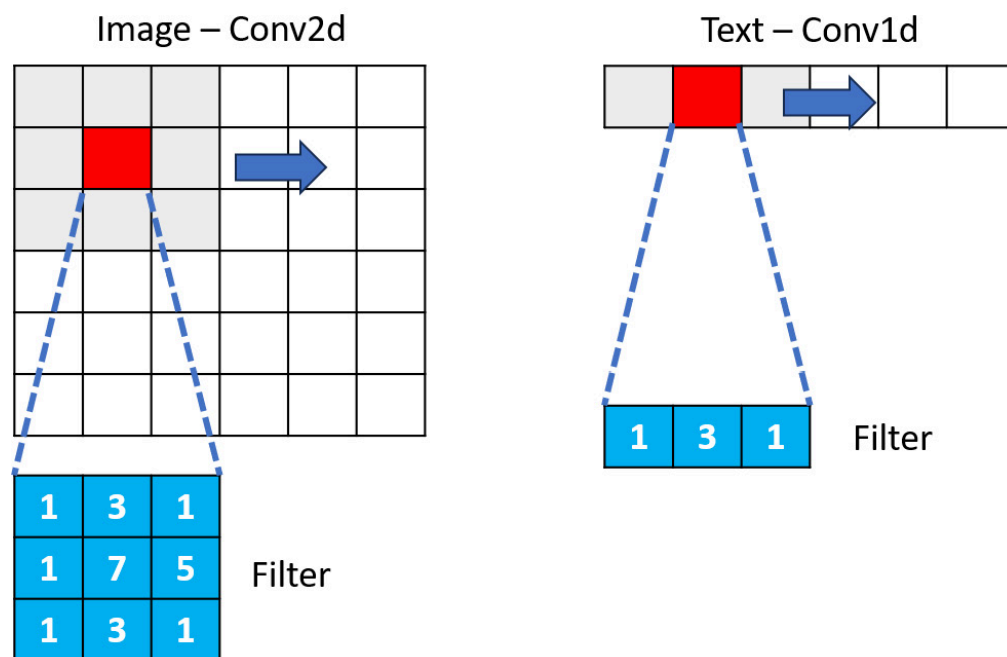


Figure 1.15 – In 1D convolution, we are sliding a 1D filter over the sequence

In the preceding figure, we scroll a uni-dimensional filter over the sequence; the process is very fast, and the filter can have an arbitrary size (three to seven words or even more). The model tries to learn patterns among the various words found within this kernel. It can also be used on vectors previously obtained from an embedding, and we can also use multiple kernels (so as to learn different patterns for each sequence). As with image CNNs, we can add operations such as max pooling to extract the most important features.

We can test in Python with a PyTorch RNN to see how it is transforming the data:

```
data_tensor = torch.tensor(np.random.random((10, 5, 3)),
                           dtype=torch.float32)
Conv1d = nn.Conv1d(in_channels=5, out_channels=16,
                   kernel_size=3, stride=1, padding=1)
```

```
output = Conv1d(data_tensor)
output.shape
```

IMPORTANT NOTE

Notice how the model is transforming the data and how this is different from what we have seen before.

Now that we have a method to transform text into numerical representation (while preserving the contextual information) and models that can handle this representation, we can combine them to obtain an end-to-end system.

Performing sentiment analysis with embedding and deep learning

In this section, we will train a model for conducting sentiment analysis on movie reviews. The model we will train will be able to classify reviews as positive or negative. To build and train the model, we will exploit the elements we have encountered so far. In brief, we're doing the following:

- We are preprocessing the dataset, transforming in numerical vectors, and harmonizing the vectors
- We are defining a neural network with an embedding and training it

The dataset consists of 50,000 positive and negative reviews. We can see that it contains a heterogeneous length for reviews and that on average, there are 230 words:

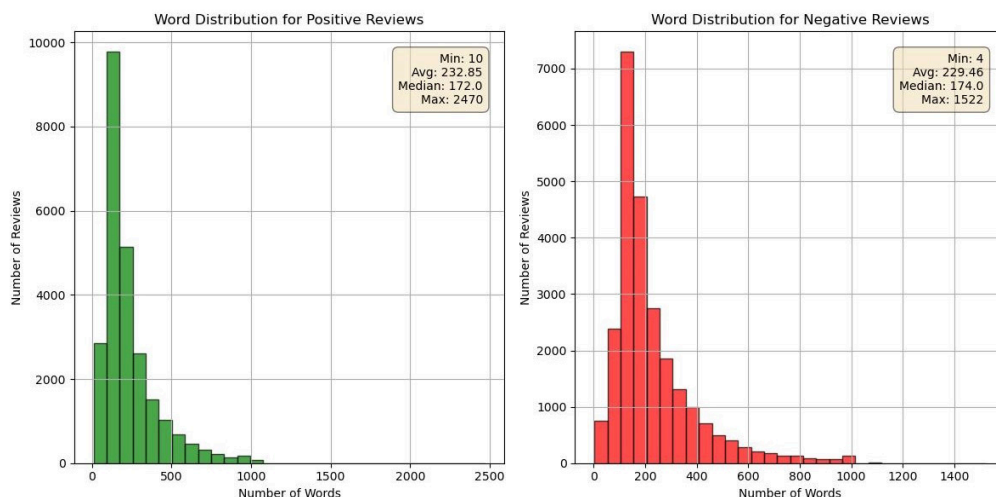


Figure 1.16 – Graphs showing the distribution of the length of the review in the text; the left plot is for positive reviews, while the right plot is for negative reviews

In addition, the most prevalent words are, obviously, “movie” and “film”:



Figure 1.17 – Word cloud for the most frequent words in positive (left plot) and negative (right plot) reviews

The text is messy and must be cleaned before the model can be trained. The first step is binary encoding of the label (“positive” equals 0, “negative” equals 1). After that, we divide the features and the labels (for a dataset in **supervised learning**, **X** are the features and **y** are the labels). Next, we create three balanced datasets for training, validation, and testing:

```
df['sentiment_encoded'] = np.where(
    df['sentiment']=='positive',0,1)
X,y = df['review'].values, df['sentiment_encoded'].values
x_train,x_test,y_train,y_test = train_test_split(
    X,y,stratify=y, test_size=.2)
x_train,x_val,y_train,y_val = train_test_split(
    x_train,y_train,stratify=y_train, test_size=.1)
y_train, y_val, y_test = np.array(y_train), np.array(y_val), \
    np.array(y_test)
```

A few steps are necessary before proceeding with the training:

- A **preprocessing** step in which we remove excessive spaces, special characters, and punctuation.
- A **tokenization** step in which we convert the various reviews into tokens. In this step, we also remove stopwords and single-character

words. We extract for each review only the 1,000 most popular words (this step is only to reduce computation time during training).

- Transformation of the the words into indices (**vectorization**) according to our vocabulary to make the model work with numerical values.
- Since the reviews have different lengths, we apply padding to harmonize the length of the review to a fixed number (we need this for the training).

These preprocessing steps depend on the dataset. The code is in the GitHub repository. Note, however, that the tokenization and preprocessing choices alter the properties of the reviews – in this case, the summary statistics.

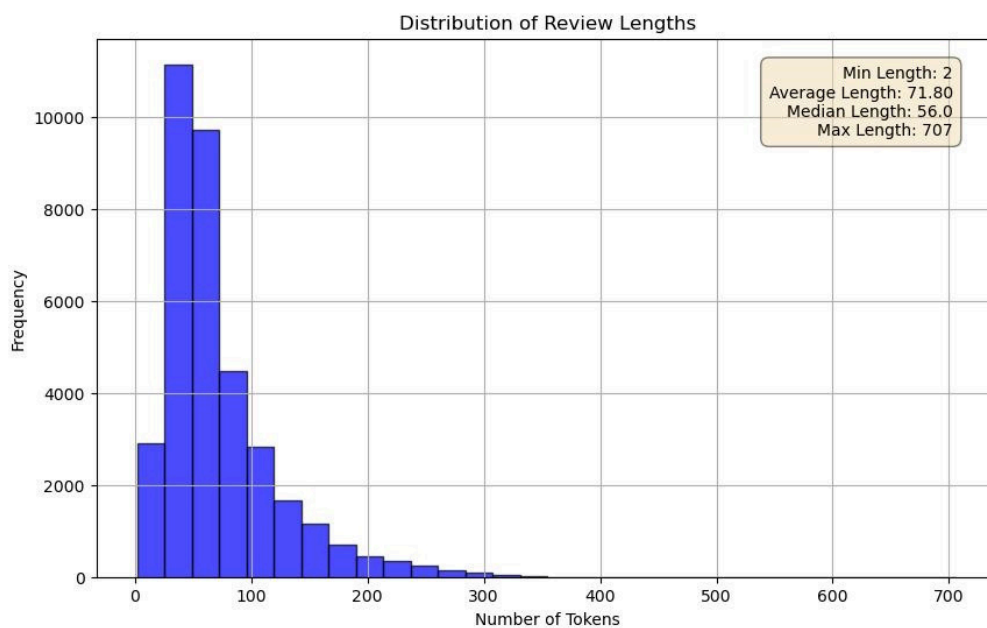


Figure 1.18 – Graph showing the distribution of review length after tokenization

We are defining the model with its hyperparameters. In this case, we are training a neural network to predict sentiment data composed of embeddings and GRUs. To make the training more stable, we add regularization (dropout). The linear layer is to map these features that we extracted to a single representation. We use this representation to calculate the probability that the review is positive or negative:

```
# Hyperparameters
no_layers = 3
vocab_size = len(vocab) + 1 # extra 1 for padding
embedding_dim = 300
output_dim = 1
hidden_dim = 256
# Initialize the model
```

```
model = SentimentRNN(no_layers, vocab_size, hidden_dim,
                    embedding_dim, drop_prob=0.5)
```

Note that in this case, we use binary cross-entropy loss because we have only two categories (positive and negative). Also, we use **Adam** as an optimizer, but one can test others. In this case, we conduct batch training since we have thousands of reviews:

```
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
epoch_tr_loss, epoch_vl_loss = [], []
epoch_tr_acc, epoch_vl_acc = [], []
for epoch in range(epochs):
    train_losses = []
    train_acc = 0.0
    model.train()
    h = model.init_hidden(50)
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        h = h.data
        model.zero_grad()
        output, h = model(inputs, h)
        loss = criterion(output.squeeze(), labels.float())
        loss.backward()
        train_losses.append(loss.item())
        accuracy = acc(output, labels)
        train_acc += accuracy
    optimizer.step()
```

The following graph displays the accuracy and loss for the training and validation sets:

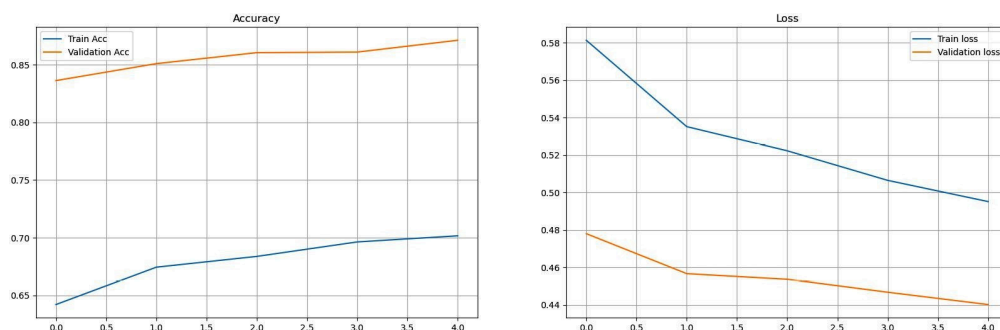


Figure 1.19 – Training curves for training and validation set, for accuracy and loss

The model achieves good accuracy, as we can see from the following confusion matrix:

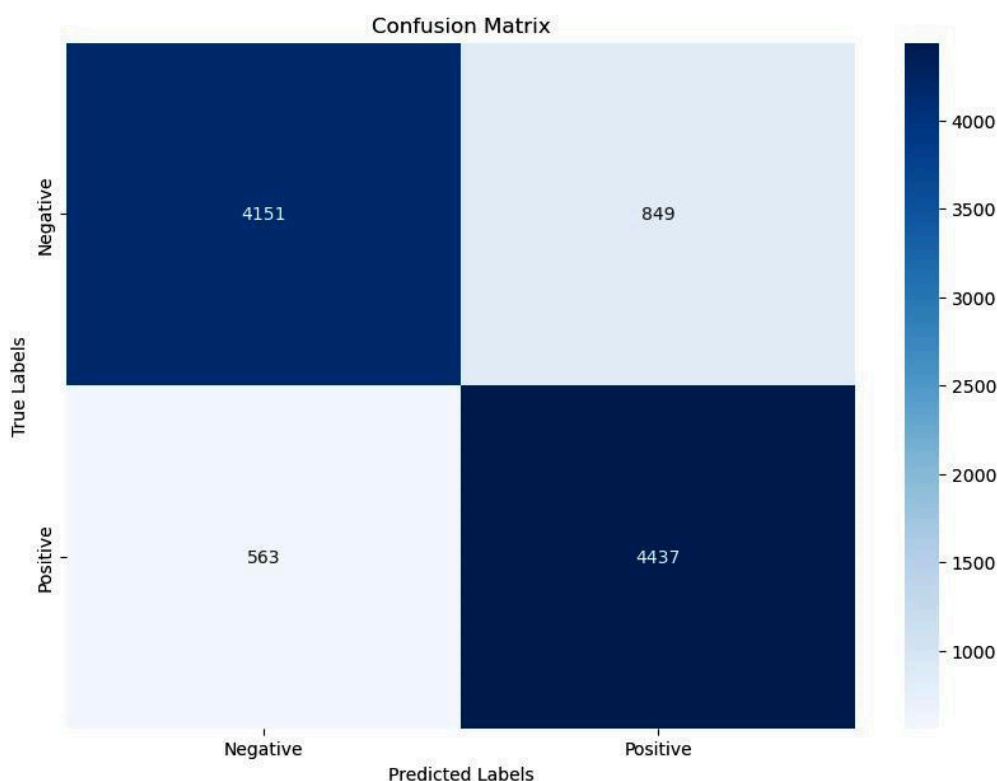


Figure 1.20 – Confusion matrix for the test set

In addition, if we look at the projection of reviews before and after the training, we can see that the model has learned how to separate positive and negative reviews:

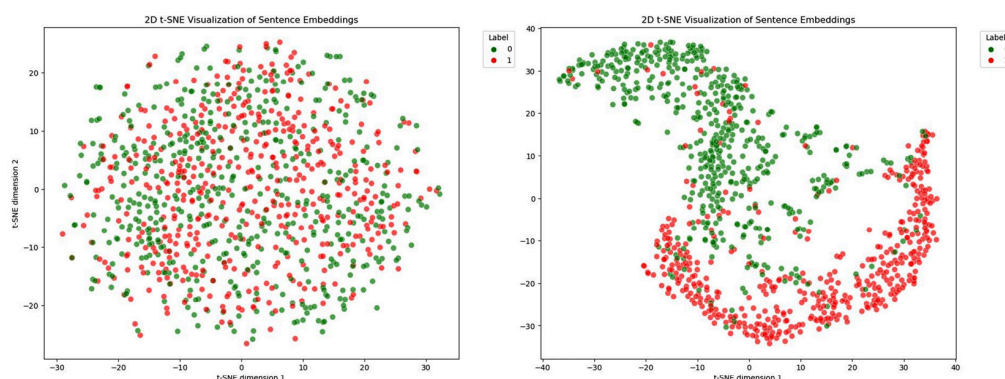


Figure 1.21 – Embedding projection obtained from the model before (left plot) and after (right plot) training

We have now trained a model that can take a review in plain text and classify it as positive or negative. We did that by combining the elements we saw previously in the chapter. The same approach can be followed with any other dataset; that is the power of deep learning.

Summary

In this chapter, we saw how to transform text to an increasingly complex vector representation. This numerical representation of text allowed us to be able to use machine learning models. We saw how to preserve the contextual information (word embedding) of a text and how this can then be used for later analysis (for example, searching synonyms or clustering words). In addition, we saw how neural networks (RNNs, LSTM, GRUs) can be used to analyze text and perform tasks (for example, sentiment analysis).

In the next chapter, we will see how to solve some of the remaining unsolved challenges and see how this will lead to the natural evolution of the models seen here.

Table of contents collapsed